

AI Resume Analyzer

Project Overview

AI Resume Analyzer is a full stack web application that analyzes resumes and provides automated feedback based on detected skills and resume content.

The application allows users to:

- Upload resumes
- Analyze technical skills
- Generate resume scores
- Receive improvement suggestions

The project was developed using React and Spring Boot.

Tech Stack

Frontend

- React
- Axios
- Vite
- HTML/CSS

Backend

- Java
- Spring Boot

AI / Analysis Approaches Explored

- OpenAI API
 - Google Gemini API
 - Ollama Local LLM
 - Lightweight Rule-Based AI Engine
-

Project Architecture

```
React Frontend
  |
  | HTTP Request
  v
Spring Boot Backend
  |
  | Resume Parsing
  | Skill Detection
  | Analysis Engine
  v
Generated Resume Feedback
  |
  v
Frontend Display
```

Folder Structure

```
AI-Resume-Analyzer
|
├── backend
|   ├── src/main/java/com/resume/backend
|   ├── src/main/resources
|   └── pom.xml
|
├── frontend
|   └── frontend
|       ├── src
|       ├── package.json
|       └── vite.config.js
|
├── README.md
└── .gitignore
```

Setup Instructions

1. Clone Repository

```
git clone https://github.com/Janakarajankumar/AI-Resume-Analyzer.git
```

2. Open Project

Open the project in:

- VS Code
 - IntelliJ IDEA
 - Eclipse
-

Backend Setup

Step 1 — Navigate to Backend Folder

```
cd backend
```

Step 2 — Run Spring Boot Application

Windows

```
.\mvnw spring-boot:run
```

Linux / Mac

```
./mvnw spring-boot:run
```

Step 3 — Backend Running

Backend runs on:

```
http://localhost:8081
```

Frontend Setup

Step 1 — Navigate to Frontend Folder

```
cd frontend/frontend
```

Step 2 — Install Dependencies

```
npm install
```

Step 3 — Start Frontend

```
npm run dev
```

Step 4 — Frontend Running

Frontend runs on:

```
http://localhost:5173
```

How to Use

1. Start backend server
 2. Start frontend server
 3. Open frontend URL in browser
 4. Upload resume file
 5. Click Analyze
 6. View generated analysis
-

Features

- Resume upload
 - Skill detection
 - Resume scoring
 - Suggestions generation
 - Frontend/backend integration
 - REST API communication
-

AI Integration Journey

1. OpenAI API Integration

Initially, OpenAI API was integrated to generate intelligent resume analysis.

Purpose

- Generate dynamic AI feedback
- Provide strengths and weaknesses
- Improve realism of resume analysis

Challenges Faced

- API billing requirements
- HTTP 429 quota errors
- API key limitations

Result

Integration worked technically, but free-tier limitations prevented reliable usage for the assignment.

2. Google Gemini API Integration

The project was later migrated to Google Gemini API.

Purpose

- Use free-tier AI API
- Reduce dependency on paid APIs

Challenges Faced

- Invalid API key issues

- Free-tier quota restrictions
- Request limitations
- JSON escaping problems

Result

Gemini integration successfully connected, but quota exhaustion and rate limits affected consistency.

3. Ollama Local LLM Integration

A local LLM approach using Ollama and Llama 3.2 was implemented.

Purpose

- Avoid external API costs
- Run AI locally
- Enable offline AI analysis

Technologies Used

- Ollama
- llama3.2 model
- OkHttp

Challenges Faced

- Long response times
- Timeout exceptions
- Large PDF text handling
- Deployment complexity for GitHub users

Result

Local AI integration worked, but performance and deployment simplicity became concerns for assignment evaluation.

4. Final Lightweight AI Engine

The final implementation uses a lightweight rule-based analysis engine.

Why This Approach Was Chosen

The rule-based AI engine was selected because it provides:

- Fast response times
- Stable execution
- No API dependency
- No billing requirement
- Easy deployment
- Consistent output

How It Works

The backend analyzes uploaded resume text and detects keywords such as:

- Java
- Spring Boot
- React
- SQL
- HTML
- CSS
- JavaScript

Based on detected skills:

- resume score is generated
- strengths are identified
- suggestions are returned

Example Analysis Logic

```
If resume contains "Java":  
+15 score
```

```
If resume contains "React":  
+15 score
```

Prompts Used During AI Experimentation

Example prompt used for LLM-based analysis:

Analyze this resume and provide:

1. Summary
2. Strengths
3. Weaknesses
4. Suggestions

What Worked Well

- Full stack integration
- REST API communication
- File upload handling
- Resume analysis workflow
- Skill detection system
- Frontend result rendering
- Multiple AI integration experiments

Problems Solved During Development

- API quota handling
- JSON escaping issues
- Backend timeout management
- Port conflicts
- Frontend/backend communication errors
- Maven dependency issues
- Local LLM integration challenges

Where AI Output Needed Manual Correction

During LLM experimentation:

- responses were inconsistent
- formatting varied
- some outputs were inaccurate
- timeouts occurred for larger resumes

The final lightweight AI engine was designed to provide more predictable and stable outputs.

Assumptions Made

- Uploaded resumes contain readable text
 - Analysis is keyword-based
 - Resume quality is estimated using detected technical skills
 - System is intended for educational/demo purposes
-

Future Improvements

- Real production-grade LLM integration
 - PDF text extraction using Apache PDFBox
 - ATS scoring system
 - Authentication system
 - Database storage
 - Resume history tracking
 - Cloud deployment
-

Author

Janakarajankumar